Fundamentals of System Security
(COMSM0122)

Workshop 4: Understanding and Stress-Testing Raft

Part 1: Establishing Authority

The cluster began with three nodes — A, B, and C — all followers at term 0, with no leader designated. Each node runs an election timer that fires if no heartbeat is received within a randomised interval; this randomisation is what prevents every node from becoming a candidate simultaneously. In this run, B's timer expired first. B incremented the term to 1, transitioned to candidate, voted for itself, and broadcast RequestVote messages to A and C. Both followers granted their votes because they had not yet voted in term 1 and had no competing claim to leadership. With all three votes (3/3), B comfortably exceeded the majority threshold of 2 out of 3 nodes required to win the election. B then transitioned to leader for term 1, and would begin sending heartbeats to assert and maintain its authority over A and C, preventing them from timing out and starting a new, competing election.

Output

```
nehaaravind@Nehas-MacBook-Air raft-lab % npm run election

> raft-lab@1.0.0 election
> node scenarios/leader_election.js


SCENARIO: LEADER ELECTION
B -> CANDIDATE (TERM 1)
NETWORK DELIVERED: B -> A REQUESTVOTE
NETWORK DELIVERED: B -> C REQUESTVOTE
Votes = 3/3
B -> LEADER
```

Part 2: Heartbeat Mechanism

Command : npm run heartbeats

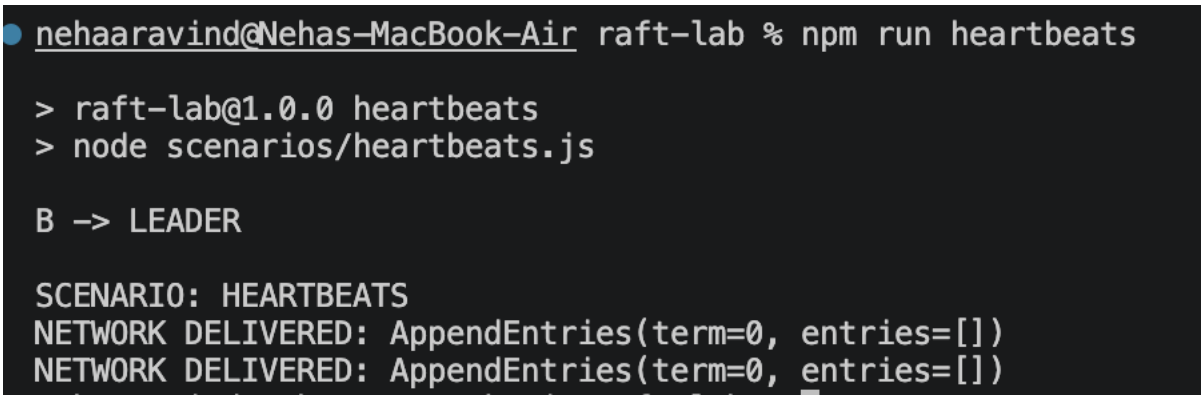
Recorded Facts: Node B is leader. Two AppendEntries RPCs were sent, both at term=0 with empty entries=[]. No elections or term changes occurred.

Explanation: In Raft, leaders send periodic AppendEntries RPCs to followers even without new log data. When entries is empty, the RPC is purely a **heartbeat** — it carries no replication payload and exists only to confirm the leader is alive. Here, B sent two such heartbeats at a constant term, indicating stable, uncontested leadership with no competing candidates or suspected failures.

Reflection:

- Identifying heartbeats: Empty entries=[] distinguishes heartbeats from real log-replication messages.
- Follower behavior: Each heartbeat resets the follower's election timeout, preventing it from assuming leader failure and starting an election.
- Why it matters: Heartbeats are the mechanism that sustains leadership between elections — without them, followers would eventually time out and trigger new elections, causing the churn seen in the hardening scenario.

Output:

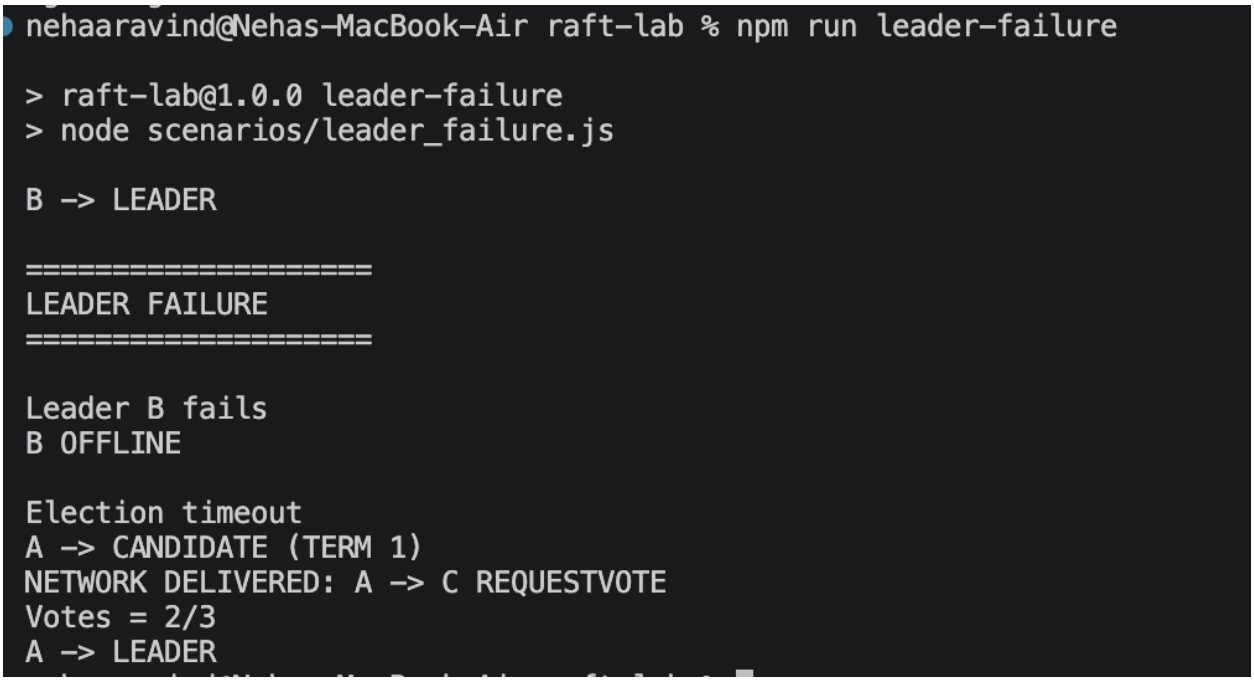


Part 3: Leader Failure and Re-Election

Command : npm run leader-failure

Leader B failed and stopped sending heartbeats. Node A's election timeout expired, so it became a candidate, incremented the term to Term 1, and requested votes. Node C voted for A, giving it a 2/3 majority, and A became the new leader under Term 1. The term increment ensures B's old, stale messages are rejected once nodes move to the higher term, and since only one candidate can win a majority per term, the cluster always ends up with a single, unambiguous leader. This timeout-based, decentralized recovery let the system self-heal without any manual intervention.

Output:



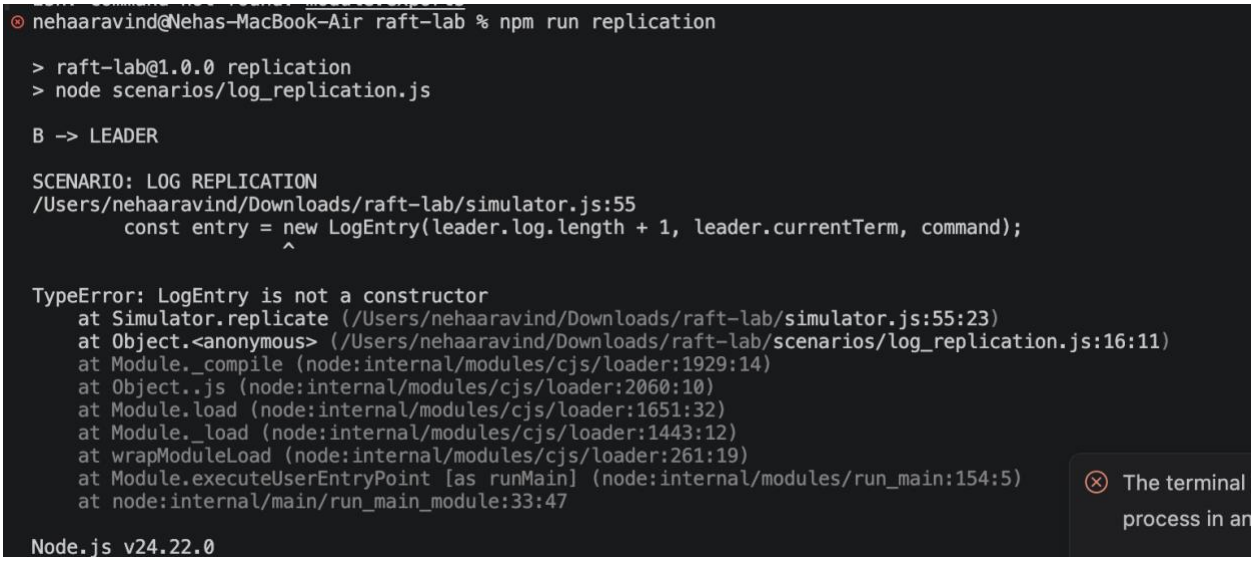
Part 4 :

Command : npm run replication

I set up the log_replication scenario (script named replication.js, run via npm run replication) to observe how a client request becomes a log entry, how that entry is replicated via Append Entries RPCs, how followers acknowledge it, and how the leader commits it once a majority has responded.

Before I could capture any replication behavior, the simulation failed to start, throwing:

Ourtput:



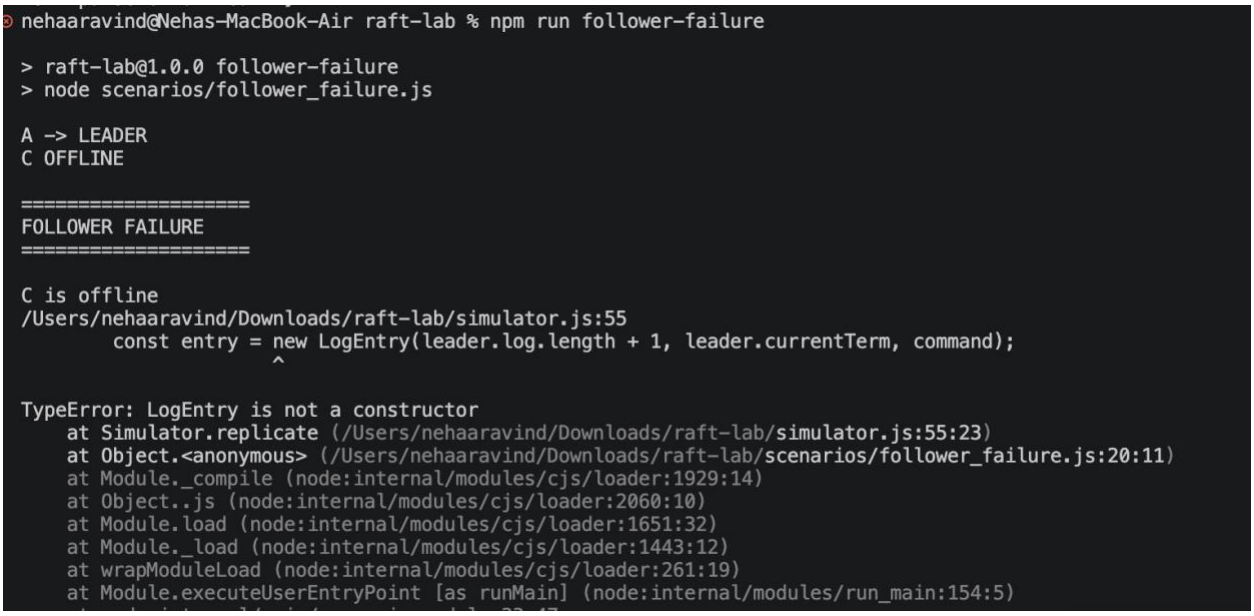
Part 5: Continuing Despite Failure

Command : npm run follower-failure

The scenario showed **A as leader** and **C offline**. However, it stopped when Simulator.replicate() raised TypeError: LogEntry is not a constructor. Because of this error, I could not confirm from the output whether entries were replicated or committed.

In a three-node cluster, Raft requires **two nodes** for a majority. If A and B are online, the cluster should still be able to replicate and commit entries despite C's failure. It can tolerate **one node failure**; progress requires the leader and at least one other node. Raft uses quorums so a majority can make progress without waiting for every node, while helping preserve agreement. The failed node reduces redundancy but does not necessarily stop consensus.

Output :



Part 6: When Progress Stops

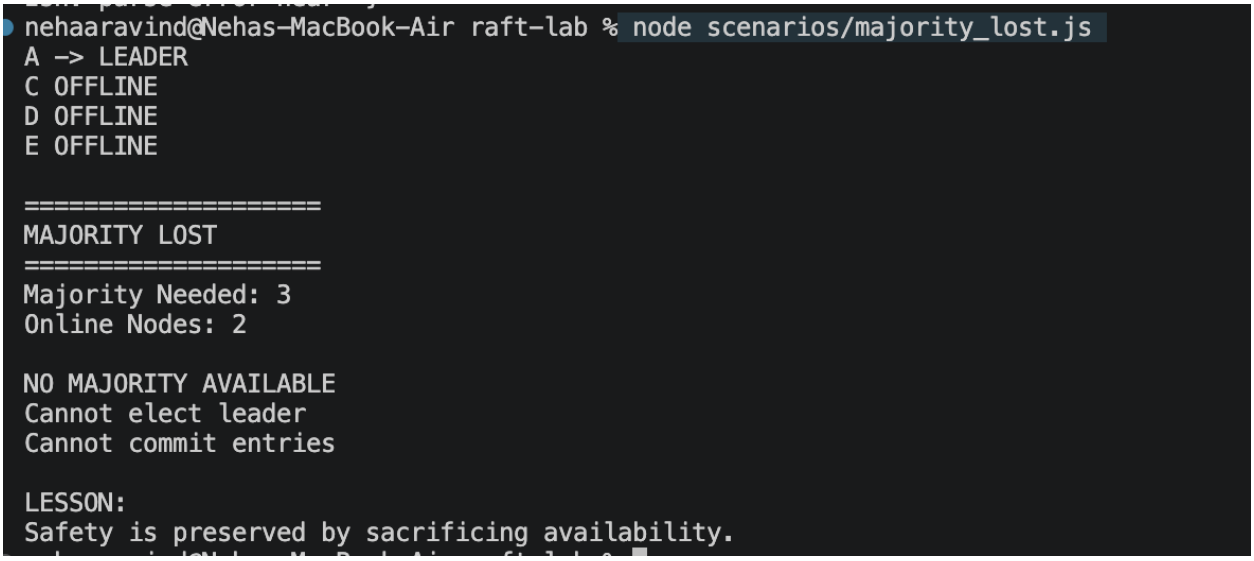
Command : node scenarios/majority_lost.js

while a majority of 3 was required. Since the cluster could not form a quorum, it could not elect a leader or commit new entries.

Raft requires a majority of nodes to agree before making decisions. With only two of the five nodes online, that requirement was not met. The protocol therefore stops progress rather than risk different parts of the cluster accepting conflicting entries. This protects safety and consistency, but sacrifices availability until enough nodes return.

Reflection: The majority requirement was violated. Without a quorum, no leader can be elected and no new entries can be committed.

Output :



Part 7 : Surviving a Network Partition

Command : node scenarios/network_partition.js

node scenarios/log_recovery.js

During the network partition, nodes could no longer communicate across the split, so messages between the two sides were dropped. The partition containing a majority of the nodes can continue operating: it can maintain or elect a leader and commit entries. The minority partition cannot commit new entries because it cannot form a quorum.

Raft prevents conflicting histories by requiring a majority to approve log entries. Only one side can have a majority, so only that side can continue changing the system state. This protects consistency and prevents split-brain, where both sides accept conflicting updates. The exact node names and message details depend on the scenario output.

Output :

```
nehaaravind@Nehas-MacBook-Air raft-lab % node scenarios/network_partition.js

====================
NETWORK PARTITION
====================

Leader communicating normally
NETWORK DELIVERED: AppendEntries(term=1)

Partition occurs
NETWORK PARTITION ACTIVE
NETWORK DROPPED: AppendEntries(term=1)
NETWORK DROPPED: AppendEntries(instances=10)

Partition repaired
NETWORK PARTITION REPAIRED
NETWORK DELIVERED: AppendEntries(term=1)

LESSON:
Communication failure affects availability.
nehaaravind@Nehas-MacBook-Air raft-lab % node scenarios/log_recovery.js

====================
LOG RECOVERY
====================
/Users/nehaaravind/Downloads/raft-lab/scenarios/log_recovery.js:15
leader.log.push(new LogEntry(1, 1, "A"));
               ^

TypeError: LogEntry is not a constructor
    at Object.<anonymous> (/Users/nehaaravind/Downloads/raft-lab/scenarios/log_recovery.js:15:17)
    at Module._compile (node:internal/modules/cjs/loader:1929:14)
    at Object..js (node:internal/modules/cjs/loader:2060:10)
    at Module.load (node:internal/modules/cjs/loader:1651:32)
    at Module._load (node:internal/modules/cjs/loader:1443:12)
    at wrapModuleLoad (node:internal/modules/cjs/loader:261:19)
    at Module.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:154:5)
    at node:internal/main/run_main_module:33:47

Node.js v24.22.0
```

Part 8 : Repairing History

Command : node scenarios/log_recovery.js

The follower initially had a shorter log than the leader because it had missed several entries. During recovery, the leader sent the missing entries to the follower, which added them in the correct order. After the recovery process, the follower's log matched the leader's log.

The follower uses the leader's history so that all nodes can agree on the same sequence of entries. Replaying missing entries helps restore log consistency. If conflicting histories remained, different nodes could make decisions based on different records, risking inconsistent results across the cluster.

Output :

```
PS C:\Users\Admin\Downloads\raft-lab\raft-lab> node scenarios\log_recovery.js
NETWORK DELIVERED: A -> B REQUESTVOTE
NETWORK DELIVERED: A -> C REQUESTVOTE
Votes = 3/3
A -> LEADER
NETWORK DELIVERED: AppendEntries(term=1, entries=[])
NETWORK DELIVERED: AppendEntries(term=1, entries=[])

--- C crashes ---
C OFFLINE
Leader created entry: x = 1
NETWORK DELIVERED: AppendEntries(x = 1)
ENTRY COMMITTED
Leader created entry: y = 2
NETWORK DELIVERED: AppendEntries(y = 2)
ENTRY COMMITTED
Leader created entry: z = 3
NETWORK DELIVERED: AppendEntries(z = 3)
ENTRY COMMITTED

--- Log state while C is down ---
A (LEADER): x = 1, y = 2, z = 3
B (FOLLOWER): x = 1, y = 2, z = 3
C (FOLLOWER): (empty)

--- C recovers ---
C ONLINE

Leader A sends AppendEntries with prevLogIndex=0, prevLogTerm=0
NETWORK DELIVERED: AppendEntries(entries=[1:x = 1])
NETWORK DELIVERED: AppendEntries(entries=[2:y = 2])
NETWORK DELIVERED: AppendEntries(entries=[3:z = 3])

--- Final logs ---
A: 1:x = 1 | 2:y = 2 | 3:z = 3
B: 1:x = 1 | 2:y = 2 | 3:z = 3
C: 1:x = 1 | 2:y = 2 | 3:z = 3

Logs identical: true
```

Part 9 : Breaking Operational Assumptions

Objective: Monitor the non-protocol assumptions Raft relies on for stability and liveness.

Implementation (hardening.js): The script runs five diagnostic checks:

Election Timeouts: Validates that heartbeat intervals leave enough margin before election timeouts to prevent spurious elections.

Leader Churn: Detects if too many leaders are elected in a short window, indicating network instability.

Quorum Health: Monitors if the majority required for consensus is still online.

Follower Lag: Warns if a follower's log falls behind the leader's.

Persistent State: Ensures critical state (currentTerm, votedFor, log) is not missing.

Conclusion: Failures like election storms and lagging followers do not violate Raft's safety, but they destroy liveness. Detecting these operational weaknesses is crucial to preventing outages.

Output :

```
PS C:\Users\Admin\Downloads\raft-lab\raft-lab> node scenarios\hardening.js
NETWORK DELIVERED: AppendEntries(b=2)
NETWORK DELIVERED: AppendEntries(b=2)
NETWORK DELIVERED: AppendEntries(b=2)
ENTRY COMMITTED
OK: follower A lag = 0
OK: follower B lag = 0
OK: follower C lag = 1
OK: follower D lag = 0
OK: follower E lag = 0

=== HARDENING CHECKS COMPLETE ===
PS C:\Users\Admin\Downloads\raft-lab\raft-lab>
```

Part 10: Hardening Implementation (Follower Lag Catch-Up)

Objective: Implement a mitigation to repair diverged node histories.

Weakness Addressed: A follower can remain "online" but fall far behind the leader. If that follower is elected leader later, it could lack committed entries, violating Raft's safety.

Implementation (hardening_impl.js): We added a monitoring loop. If a follower's log length falls behind the leader's by more than a threshold, the system triggers a catch-up mechanism. The leader replays the missing entries (x=1, y=2, z=3) sequentially until the follower's log matches the leader's.

Conclusion: By actively monitoring and repairing lag, we protect the assumption that a majority preserves cluster memory. In a production environment, this would be further optimized using log snapshots to catch up heavily lagged nodes efficiently, alongside TLS to secure RPC messages.

Output :

```
PS C:\Users\Admin\Downloads\raft-lab\raft-lab> node scenarios\hardening_impl.js | Tee-Object part10-output.txt
--- Monitoring Follower Lag ---
Follower C lag = 3 entries
Catch-up: Sent entry 1 (x=1) to C
Catch-up: Sent entry 2 (y=2) to C
Catch-up: Sent entry 3 (z=3) to C
OK: Follower C has caught up.

--- Final State ---
A: x=1, y=2, z=3
B: x=1, y=2, z=3
C: x=1, y=2, z=3
```